

Loop Engineering

From prompting to looping: designing the systems that prompt your agents.

AKINROBOTICS Tech Talks - Ahmet Ataşoğlu - 10.07.2026

You shouldn't be prompting coding agents anymore. You should be **designing loops** that prompt your agents.

// PETER STEINBERGER · CREATOR OF OPENCLAW · JUNE 2026

8.9M VIEWS IN DAYS

AND THE RECEPTION SPLIT HARD

39% mindset shift

61% bad advice

I The week three people lit the fuse

In June 2026, three people, not coordinating, said almost the same thing within days. A week later Andrew Ng placed it inside the wider 0-to-1 product picture.



JUN · POST

Peter Steinberger

"Stop prompting. Start designing loops that prompt your agents."



JUN · TALK

Boris Cherny

"I don't prompt Claude anymore. I have loops that prompt Claude."



JUN · ESSAY

Addy Osmani

"Replace yourself as the person who prompts the agent."



JUL · FRAME

Andrew Ng

Nests it as three loops for 0-to-1 product building.

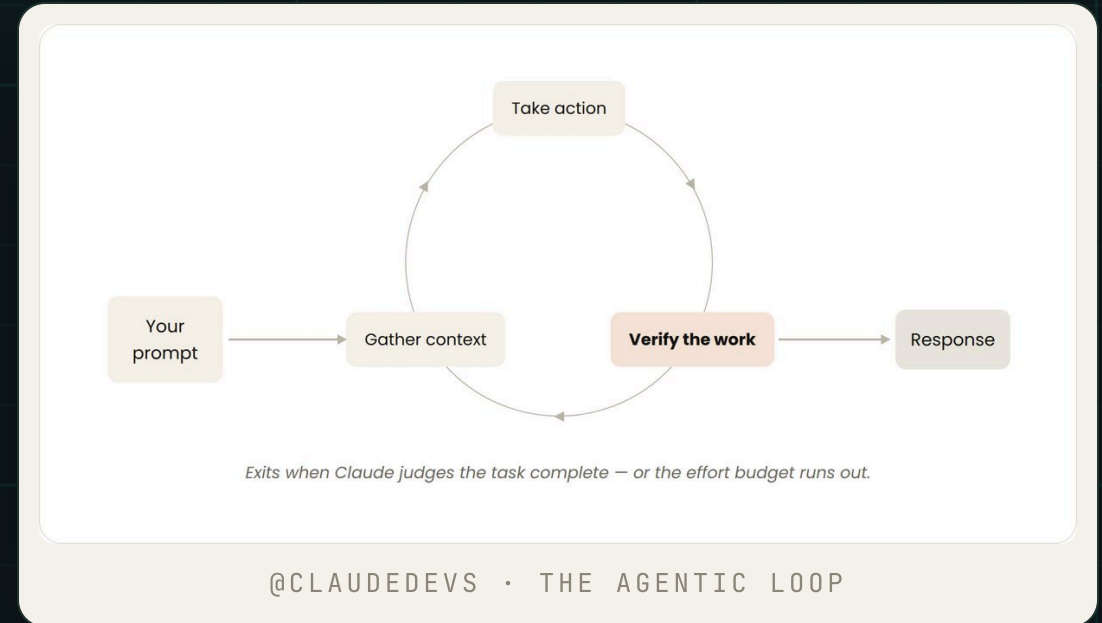
Practice preceded the name by months. Everyone was already writing loops; nobody had called it anything yet.

What a loop actually is

Agents repeating cycles of work until a stop condition is met. That is the Claude Code team's definition, verbatim.

Inside one turn: **act, observe, reason, repeat**. Two moves turn that cycle into a loop rather than a one-shot: **verification** and **scheduling**.

The property that makes it a loop, not the same task run N times: today's output becomes tomorrow's input. It feeds itself.

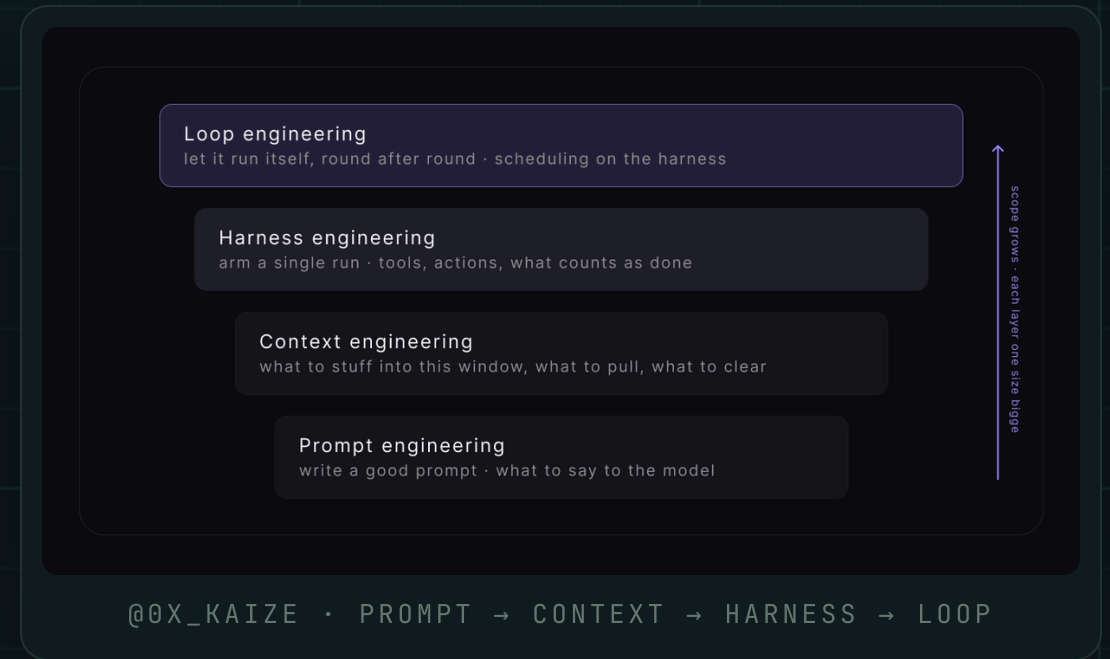


| One floor above the harness

Four rungs, each one size bigger than the last:

- 01 **Prompt** engineering: optimize a single turn.
- 02 **Context** engineering: optimize what the model sees.
- 03 **Harness** engineering: design the environment one agent runs inside.
- 04 **Loop** engineering: adds three verbs the harness lacks. It runs on a timer, spawns helpers, and feeds itself.

Same bug costs more at each layer: a loop is a machine built to make "number of turns" large.



I Why 2026 and not 2024

2024 · HUMAN IN EVERY TURN

Coherent task horizons of about an hour. You babysat each step. The bottleneck was the model.

2026 · HUMAN AT THE EDGES

Opus-class horizons of 10+ hours. Primitives ship inside the products. The bottleneck moved to loop design.

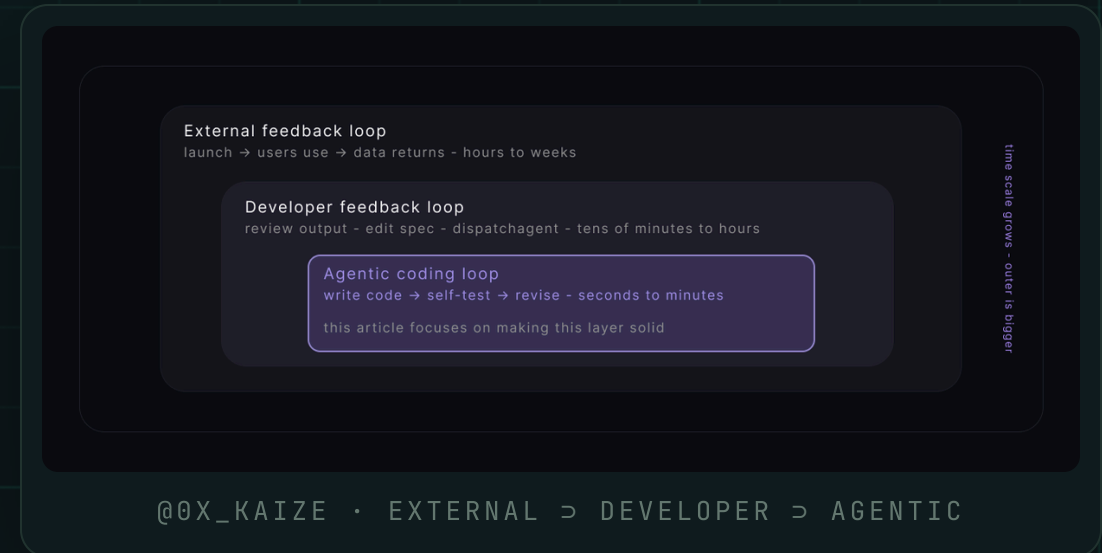
Anthropic put a number on closing the loop: a solo agent gave **20 min for \$9**; a full harness gave **6 h for \$200** on the same prompt, with the quality gap immediately visible.

| Andrew Ng: three nested loops

Nested by timescale, each outer loop consuming the inner one's output:

- **Agentic** (minutes): agent writes, tests, iterates until the spec is met.
- **Developer** (minutes to hours): you review and steer; QA shrinks as agents self-test.
- **External** (days to weeks): alpha users and A/B tests inform the vision, which drives the spec.

Ng's sharp point: humans stay in the loop for a **context advantage**, not taste. "So long as the human knows something the AI does not," you inject that knowledge.



01

The Loop Taxonomy

Four kinds of loop, sorted by
what you hand off.

Four loop types

LOOP	YOU HAND OFF	USE IT WHEN	REACH FOR
Turn-based	The check	You're exploring or deciding	Verification skills
Goal-based	The stop condition	You know what done looks like	<code>/goal</code>
Time-based	The trigger	Work happens on a schedule	<code>/loop</code> , <code>/schedule</code>
Proactive	The prompt	Work is recurring, well-defined	Dynamic workflows

Each row hands off one more piece of the human's job. That progression *is* the story: you climb it as trust in the loop grows.

| The same four, as commands you'd actually type

Turn-based

"Create a like button." You watch, you verify each turn.

Goal-based

```
/goal get Lighthouse to 90+, stop after 5 tries
```

Time-based

```
/loop 5m check my PR, address review comments, fix CI
```

Proactive

Scheduled triage + goal + workflows, running whether the laptop is open or not.

Engineers trust concrete commands. The type names are just where each command sits on the hand-off ladder.

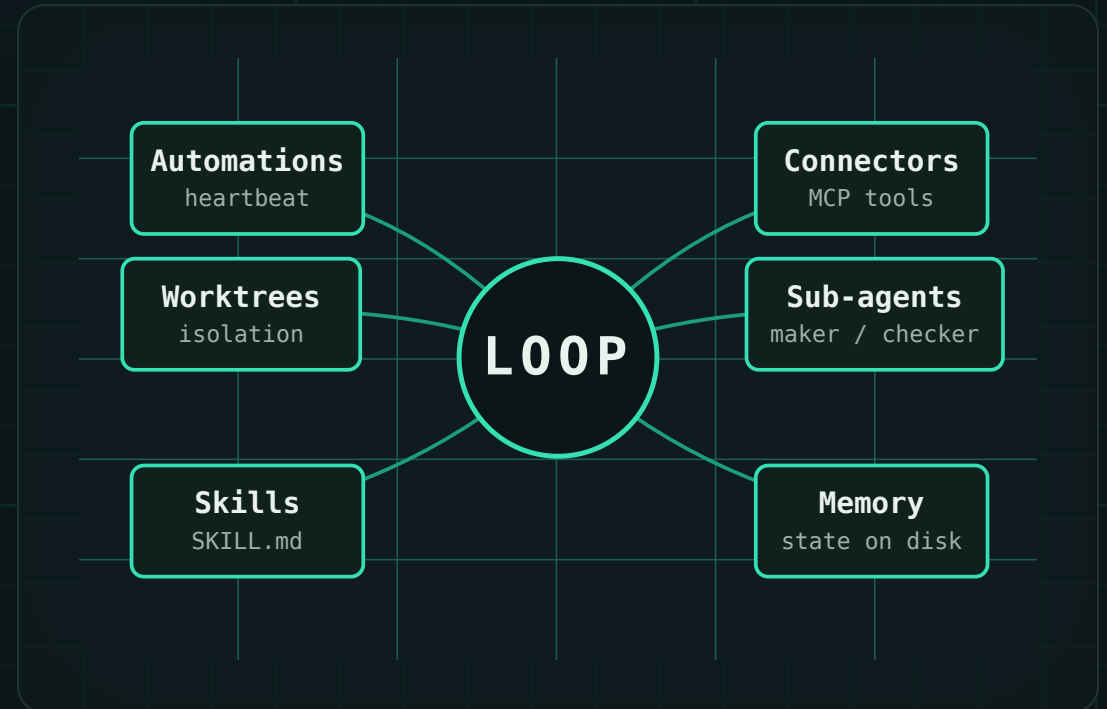
02

Anatomy of a Loop

Whatever tool you use, a loop needs five pieces and one place to remember.

What a loop is made of

- **Automations:** the heartbeat. Scheduled discovery and triage.
- **Worktrees:** isolation so parallel agents don't collide.
- **Skills:** written-down project knowledge (`SKILL.md`). Without them, the loop re-derives your project every cycle.
- **Connectors (MCP):** the loop touches real tools. Opens the PR, updates the ticket, pings Slack.
- **Sub-agents:** the maker and checker split.
- **Memory:** state on disk. The agent forgets, the repo doesn't.



I The pair everyone confuses

CONTEXT

The window the agent sees *this turn*. Cleared on every refresh. Ephemeral by design.

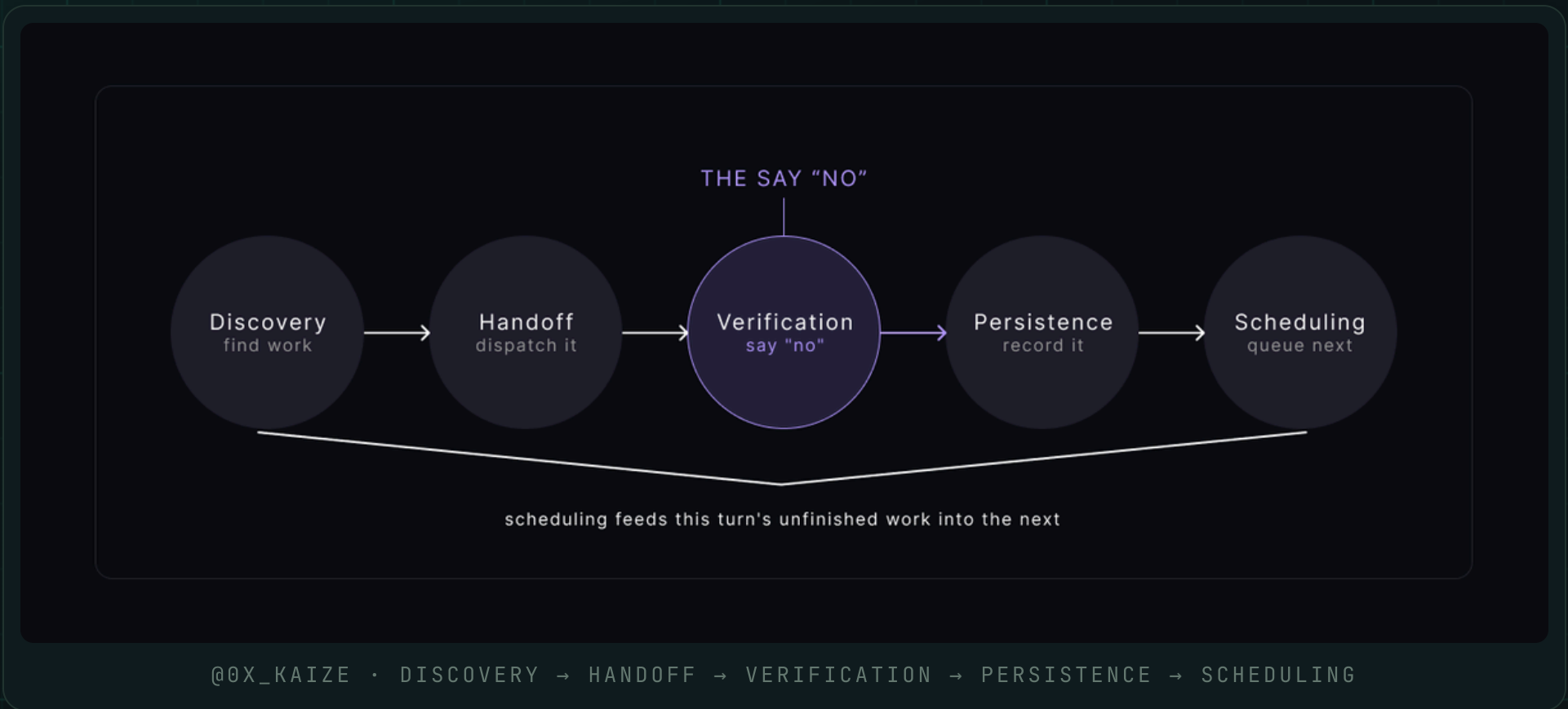
MEMORY

State that survives across turns and days. Written to disk: markdown, a DB, a board. A loop bridges today to tomorrow through memory only.

```
# ./state/triage.md
| finding      | source      | pri | status |
|-----|-----|-----|-----|
| auth 500     | ci #4821   | P1  | fixing |
| flaky e2e    | run #77    | P3  | queued |
```

No source, can't audit. No priority, can't order. No status, tomorrow can't pick up the baton.

A loop is five moves in a ring



Move 1: a morning-triage skill

Monday 6am, laptop closed. A GitHub Action fires

```
claude --skill morning-triage .
```

- **Read** names concrete sources, down to the command. "Read the context" is not a source.
- **Judge** is the ceiling of the whole loop: it decides actionable-vs-noise *itself*. Hand it a list every morning and you automated the doing, not the choosing.
- **Stop** is the only non-boilerplate part: what it must never do alone.

```
# morning-triage · SKILL.md
## Read
gh run list --status failure
gh issue list --search "created:>=1d"
git log --since=yesterday
# + yesterday's ./state/triage.md

## Judge
Decide actionable vs noise YOURSELF.

## Stop
Never merge. Never push to main.
Uncertain -> ./inbox/ for a human.
```

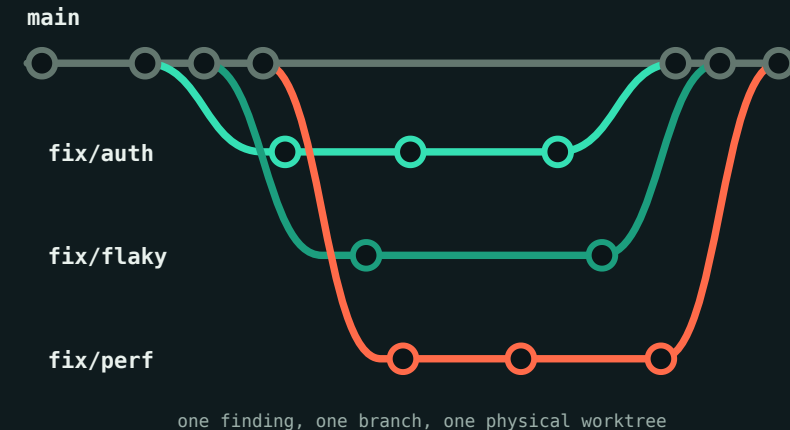
Move 2: one worktree per finding

Five findings, five agents, one directory: merge day becomes archaeology.

The antidote is physical isolation:

```
git worktree add ../fix-auth fix/auth
```

- **Never work on main.** Main is the landing strip for human review, not the agents' desk.
- **Cap the parallelism.** Not "how many can my machine run," but "how many PRs can I actually review." Review bandwidth is the ceiling.



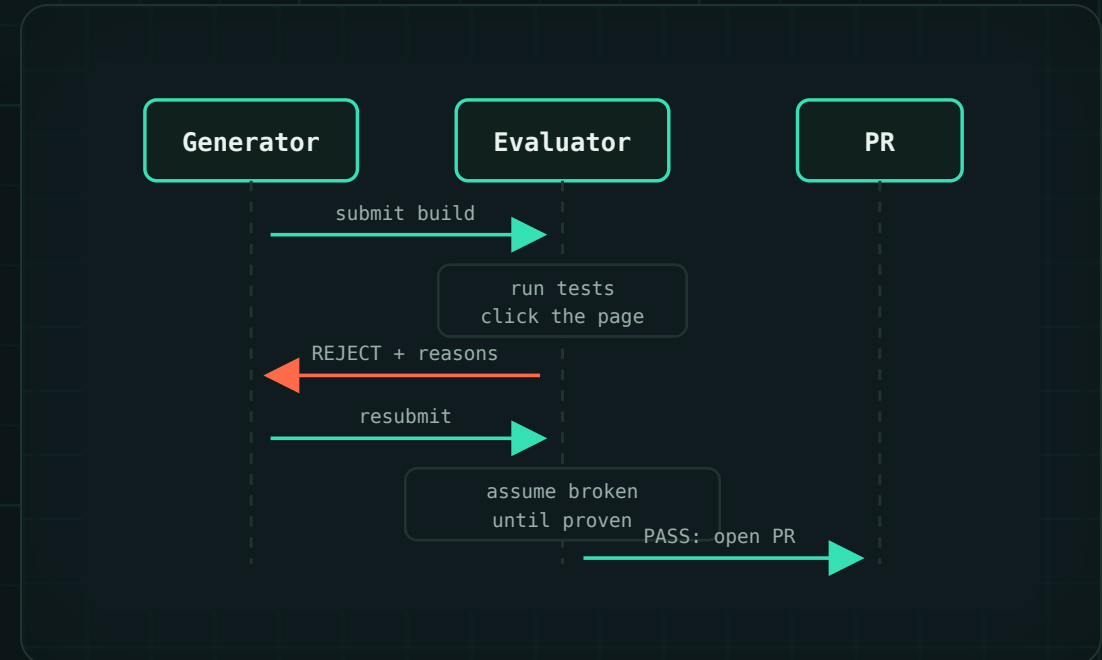
Move 3: a door that says "no"

Ask an agent to grade code it just wrote and it praises itself: its context is stuffed with the argument *for* the result, not the result.

Tuning a standalone evaluator to be skeptical is far more tractable than making a generator critical of its own work.

Prithvi Rajasekaran · Anthropic

- Two agents, ideally **two models**: same model, same blind spots.
- Default posture is **ASSUME BROKEN**; trust is earned.
- **Judge behavior, not intent**: run the tests, click the page.
- **Reject with reasons**, or the generator has nothing to fix.

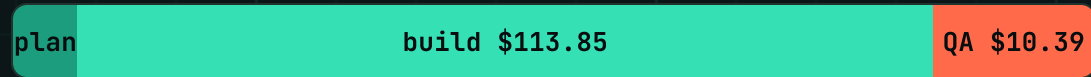


Acid test: has your evaluator rejected anything in the last 5 turns? A loop that never says "no" is decoration.

Anthropic ran this in production

A three-agent harness (planner, generator, evaluator) built a working DAW. The evaluator drove a live browser and graded against negotiated sprint contracts.

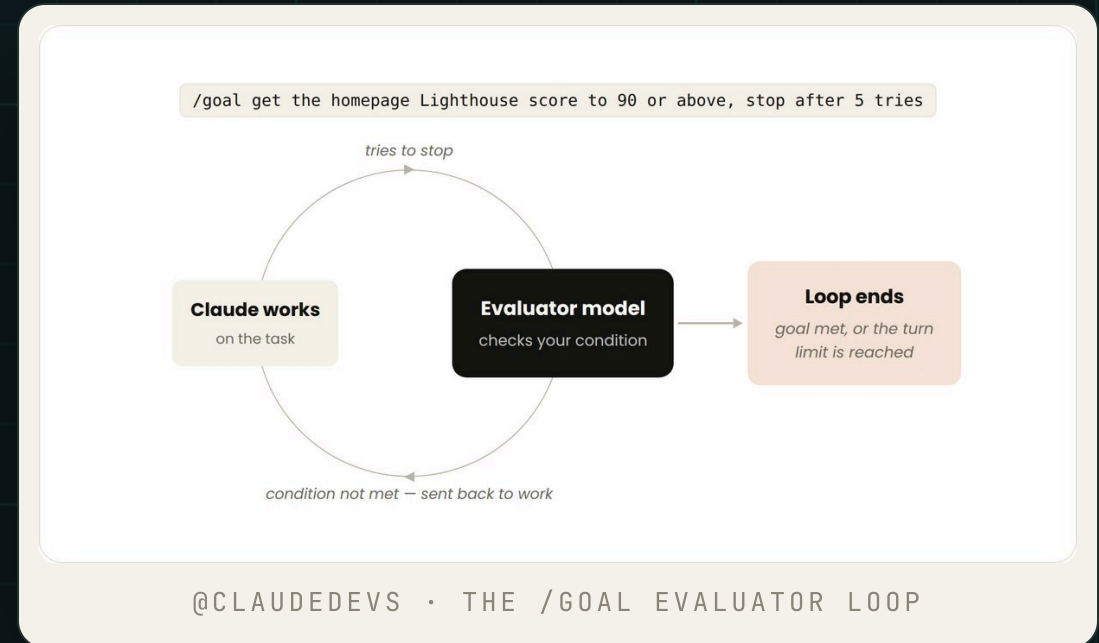
3h 50m · \$124.70 total



PLANNER \$0.46 · BUILD \$113.85 · QA \$10.39

Caught: a FastAPI `reorder` endpoint matched as a frame ID.

Caught: a fill tool placing tiles only at drag endpoints.



Worth the cost when the task sits beyond what the model does reliably solo. Not a fixed yes-or-no.

Survive until tomorrow, wake up without you

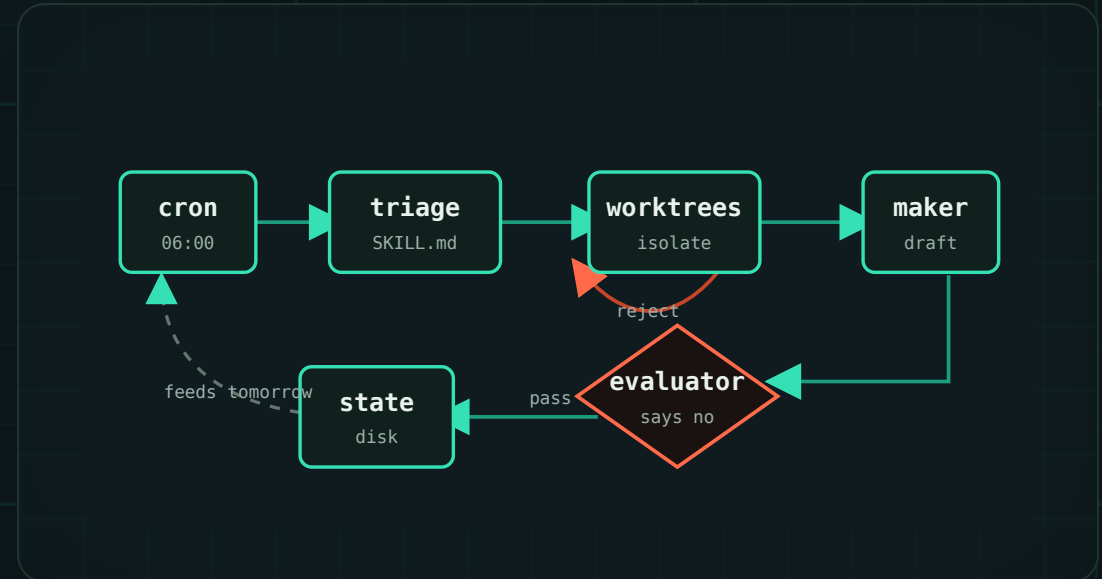
Persistence: verdicts must outlive the context window.

PASS opens a PR, REJECT updates the ticket.

Tomorrow's run picks up where today stopped.

Scheduling: the deciding question is *can this work leave your laptop?* `/loop` runs on your machine; `/schedule` moves it to the cloud.

Set three caps **before** the first autonomous run: per-run budget, daily budget, max retries. Uber capped engineers at **\$1,500 per tool per month** after burning a year's AI budget in four. Your per-loop cap is that lesson, made granular.



| Five ways loops die

DISEASE	SYMPTOM	MISSING MOVE	FIX
Blind Loop	Never surprises you	Discovery	Put <i>Judge</i> inside the skill
Tangled Loop	Five diffs collide in one file	Handoff	One worktree per task
Nodding Loop	Every turn self-approves	Verification	Separate, skeptical evaluator
Amnesiac Loop	Every day starts from zero	Persistence	State file on disk
Manual Loop	"Last run was demo day"	Scheduling	A trigger you don't own

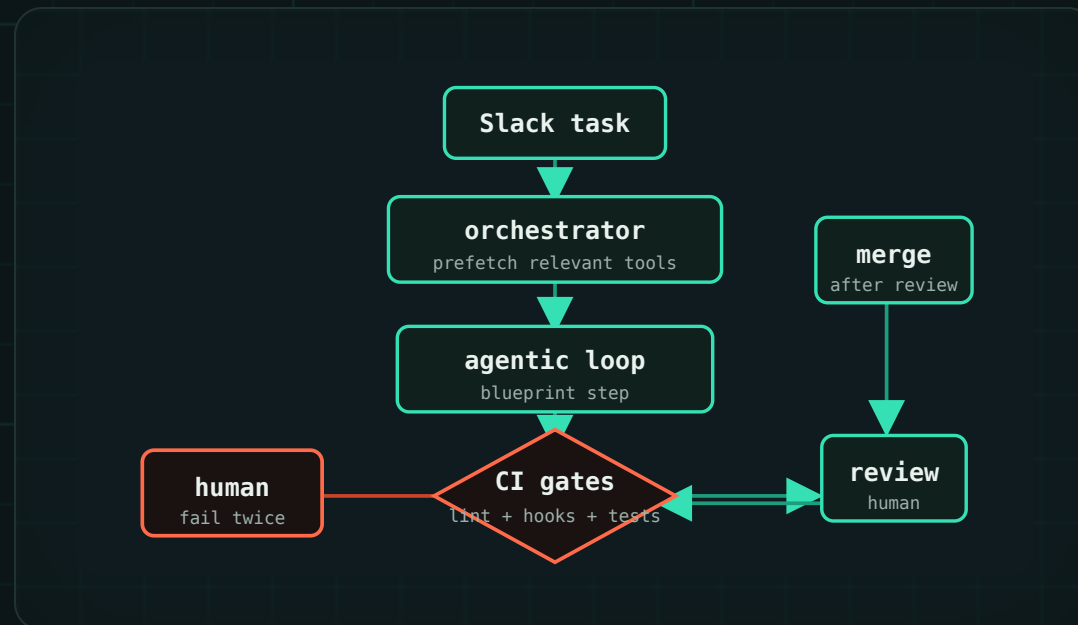
The **Nodding Loop** is the most expensive: plausible mistakes stack at machine speed, and it compounds silently.

| 1,300 PRs a week,
zero human-written code

1,300+

MERGED PRS / WEEK, ALREADY PASSING TESTS

- Blueprints alternate deterministic code and agentic loops.
- Context prefetch curates ~15 relevant tools before the LLM wakes up.
- Shift-left gates: linters, pre-push hooks, one pass through a 3M-test suite. Fail twice, a human takes over.
- Review is moved, not removed. Strict deterministic gates mattered more than raw model power.

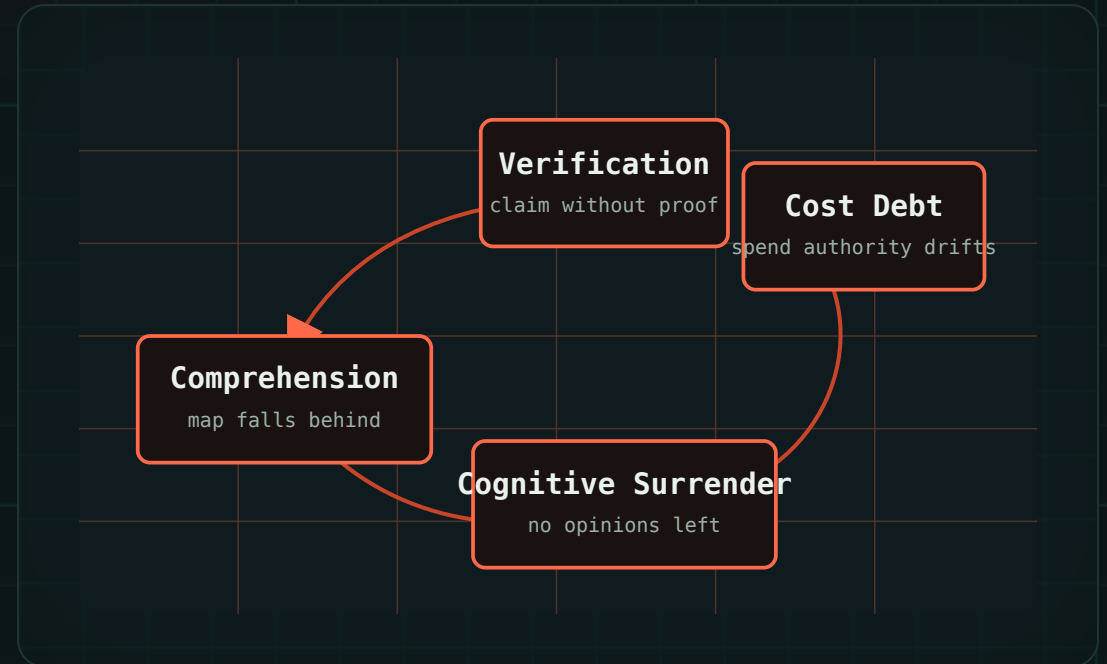


I What piles up quietly

None of these trips an alarm:

- **Verification debt:** "done" is a claim, not a proof.
- **Comprehension debt:** the gap between what exists and what you understand.
- **Cognitive surrender:** you stop having opinions.
- **Cost debt:** spend authority outsourced to bugs.

They feed each other. The daily discipline: **read one sample PR a day** and explain what changed and why. Fixing a stale mental map on a calm morning costs ten minutes; during an incident, far more.



Take the skeptics seriously

THE SKEPTICS SAY

"Isn't this just agentic design with a new name?" You are not wrong: the scheduling layer is just cron, and one stochastic model reviewing another is not automatically a control system. 61% of replies called it premature.

WHAT'S ACTUALLY NEW

Not any single piece. It's that the pieces now **ship inside the products** and the models sustain **10+ hour horizons**. The bottleneck moved from capability to loop design. That part is real.

Osmani himself: token costs vary wildly, it's early, direct prompting still works. It's about finding the balance.

Build the loop. But build it like someone who intends to **stay the engineer**, not just the person who presses go.

// ADDY OSMANI

Two people can build the exact same loop and get opposite results. The loop doesn't know the difference. You do. Start Monday: pick one task where you're the bottleneck, write the verification check. That's Move 1.

I References

- **Claude Code team** (@ClaudeDevs): "Getting started with loops." X Article, Jul 2026.
- **kaize** (@0x_kaize): "Loop Engineering: From Prompting to Looping." X Article, Jul 2026.
- **Addy Osmani**: "Loop Engineering." addyosmani.com, Jun 2026.
- **Peter Steinberger**: original post. X, Jun 2026.
- **Andrew Ng**: "3 Key Loops for 0-to-1 Products." The Batch, Jun 2026.
- **Prithvi Rajasekaran** (Anthropic): "Harness design for long-running application development." anthropic.com/engineering.
- **Stripe Minions**: ByteByteGo, MindStudio, Lenny's Newsletter coverage.
- **Uber AI budget cap**: TechCrunch, Simon Willison.